CS 6050 · GRADUATE ALGORITHM PROJECT 2

Breaking Paragraphs into Lines

The Knuth-Plass line breaking algorithm

Jacob Beal

Knuth & Plass, Software: Practice and Experience, 1981

THE PROBLEM

Where should each line of a paragraph end?

One line at a time (first fit): 13,218 demerits

(a) In olden times when wishing still helped one, there lived a king -.741
whose daughters were all beautiful; and the youngest was so .877
beautiful that the sun itself, which has seen so much, was aston- -.425
ished whenever it shone in her face. Close by the king's castle lay -.816
a great dark forest, and under an old lime-tree in the forest was -.191
a well, and when the day was very warm, the king's child went .107
out into the forest and sat down by the side of the cool fountain; -.538
and when she was bored she took a golden ball, and threw it up -.234
on high and caught it; and this ball was her favorite plaything. .000

Whole paragraph at once (Knuth-Plass): 4,030 demerits

(c) In olden times when wishing still helped one, there lived a .780
king whose daughters were all beautiful; and the youngest was .246
so beautiful that the sun itself, which has seen so much, was .667
astonished whenever it shone in her face. Close by the king's .514
castle lay a great dark forest, and under an old lime-tree in the .027
forest was a well, and when the day was very warm, the king's .173
child went out into the forest and sat down by the side of the .346
cool fountain; and when she was bored she took a golden ball, .275
and threw it up on high and caught it; and this ball was her .593
favorite plaything. .002

Figure 4 and demerits table: Knuth & Plass 1981, pp. 1129-1130

INPUT

A paragraph, a column width, and the places a word can be hyphenated

OUTPUT

The line breaks that keep the spacing as even as possible

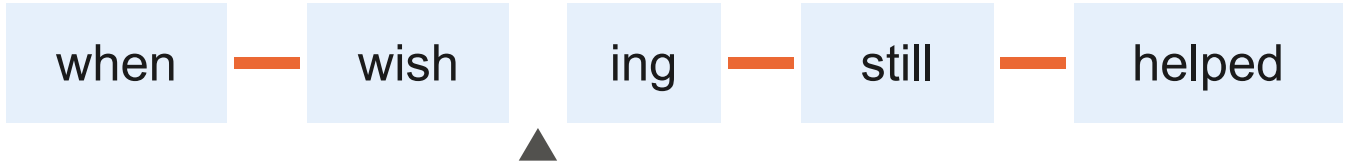
2^{129}

ways to break the paper's sample paragraph, so we can't just try them all

It sounds like bin packing, which is NP-complete. But each line holds text in order, and that is what makes DP work.

THE PROBLEM

A paragraph is just boxes, glue and penalties



Box

A word or a piece of one. It only has a width.

w = 41 for "when"

Glue

A space between words. It has a normal width and can stretch or shrink.

(width, stretch, shrink) = (6, 3, 2)

Penalty

A spot where a line could break, like a hyphen, with a cost for breaking there.

p = 50 for a hyphen

HOW MUCH DOES A LINE HAVE TO STRETCH?

$$r = (width - L) / Y$$ if the line is too short

$$r = (width - L) / Z$$ if the line is too long

L = natural width of the line, Y = total stretch, Z = total shrink

WHICH LINES ARE ALLOWED?

$r < -1$: spaces would have to shrink past their limit, so it never fits

$r > \rho$: too loose. ρ is a tolerance you pick (the paper uses 1.26)

Knuth & Plass, "Breaking Paragraphs into Lines," Software: Practice and Experience, 1981

THE PROBLEM

Every line gets a score, and we want the lowest total

DEMERITS FOR ONE LINE

$$\text{badness} = 100\,|r|^3$$

$$\text{demerits} = (1 + \text{badness} + p)^2$$

p is the penalty for the break at the end of the line

PLUS, FOR PAIRS OF LINES

+ α if two lines in a row end in a hyphen

+ γ if a tight line sits next to a loose one

Negative penalty: $(1 + \text{badness})^2 - p^2$. Forced break: $(1 + \text{badness})^2$. The paper uses $\alpha = \gamma = 3000$.

r	DEMERITS
0.00	**1**
0.25	**7**
0.50	**182**
1.00	**10,201**

FITNESS CLASSES

0	tight	$r < -0.5$
1	normal	-0.5 to 0.5
2	loose	0.5 to 1
3	very loose	$r > 1$

Classes more than 1 apart are the ones that pay γ.

Knuth & Plass, "Breaking Paragraphs into Lines," Software: Practice and Experience, 1981

THE SUBPROBLEMS

Each DP entry is the cheapest way to reach a break

D[b, c] The fewest total demerits to set everything from the start of the paragraph through break b, if the line ending at b has fitness class c.

b A legal place to end a line. There are B of them.

c Tight, normal, loose or very loose, so 4 values.

Start D[0, 1] = 0. Nothing set yet, treated as a normal line.

Answer The smallest D[m, c], where m is the forced break at the very end.

EACH ENTRY ALSO STORES

A pointer to the break it came from, which is how we get the lines back at the end

WHY c IS IN THE TABLE

The next line's cost depends on how tight this one was, so the break alone isn't enough to know what comes next

Knuth & Plass, "Breaking Paragraphs into Lines," Software: Practice and Experience, 1981

THE RECURRENCE

Try every place the last line could start

$$D[0, 1] = 0$$

$$D[b, c] = \min \text{ over active breaks } a \text{ and classes } c' \text{ of}$$

$$D[a, c'] + \text{demerits}(a \to b) + \alpha \cdot (a, b \text{ both hyphens}) + \gamma \cdot (|c - c'| > 1)$$

only where the line from a to b fits ($-1 \le r \le \rho$) and its class is c

READING IT OFF

D[a, c′]

best way to set everything before this line

demerits(a → b)

this line's own cost, $(1 + 100|r|^3 + p)^2$

α and γ

cost of this line next to the line before it

r comes from running totals of width, stretch and shrink, so each check is O(1). With line numbers in the state, the same idea handles different line lengths.

Knuth & Plass, "Breaking Paragraphs into Lines," Software: Practice and Experience, 1981

THE RECURRENCE

Why the best answer has to be one of those cases

1

Every setting has a last line

It starts right after some earlier break a, or at the start. Trying every a covers every possible setting.

2

What came before only matters through (a, c′)

Later lines only see where the last break was and how tight that line was. Swap in a cheaper way to reach (a, c′) and the total can only go down.

3

Throwing out a break is safe

If a line from a is already too long ($r < -1$), it only gets longer for every later break, so a can never start another line.

WHY c HAS TO BE THERE

Keep only the cheapest way to reach each break and #2 stops being true. A costlier way in can win later by avoiding γ.

40 of 148,000

random paragraphs got a worse answer when I dropped the classes

Rare enough that random tests missed it, so one of them is now a unit test.

Knuth & Plass, "Breaking Paragraphs into Lines," Software: Practice and Experience, 1981

THE RECURRENCE

Only a short list of breaks stays in play

```
for each legal break b:
    for each active node a:
        r = ratio(a, b)
        if r < -1: remove a
        if -1 <= r <= rho:
            update D[b, class(r)]
    add the best entries for b
      to the active list
```

A break stays active only while a line starting there can still fit. So the list is about one line of text long, and it does not grow with the size of the paragraph.

O(B · W)

B breakpoints, each checking about W active nodes, where W is words per line

17

active breaks at one point in the paper's example, with 836,272,858 possible settings

Each break can have up to 4 nodes, one per class. That is hidden in the constant, and it shows up in my results later.

Running time and 17 active nodes: Knuth & Plass 1981, pp. 1148, 1151

A WORKED EXAMPLE

A small example with 8,192 possible settings

THE INPUT (WORD WIDTHS UNDERNEATH)

when	she	was	bored	she	took	a	golden	ball,	and	threw	it	up	on
41	25	29	44	25	35	9	51	34	29	45	12	20	19

COLUMN WIDTH

236 units

SPACES

(6, 3, 2)

TOLERANCE

ρ = 1

CONTRAST COST

γ = 3,000

ONE LINE BY HAND

"when she was bored she took"

L = 199 + 5 × 6 = 229, which is 7 short

Y = 5 × 3 = 15, so r = 7 / 15 = 0.467

$(1 + 100 \cdot 0.467^3)^2 = 124.6$ demerits

3

settings where every line fits

4

breaks are ever reachable: "took", "a", "up", "on"

A WORKED EXAMPLE

Filling in the table, one break at a time

BREAK b	FROM a	LINE	L	r	CLASS	DEMERITS	γ	D[b, c]
took	start	when she was bored she took	229	+0.467	1	124.6		**124.6**
a	start	when she was bored she took a	244	-0.667	0	938.2		**938.2**
up	took	a golden ball, and threw it up	236	+0.000	1	1.0		**125.6**
up	a	golden ball, and threw it up	221	+1.000	2	10,201.0	3,000	14,139.2
on	a	golden ball, and threw it up on	246	-0.833	0	3,465.7		4,403.9
on	up	on	19	+0.002	1	1.0		**126.6**

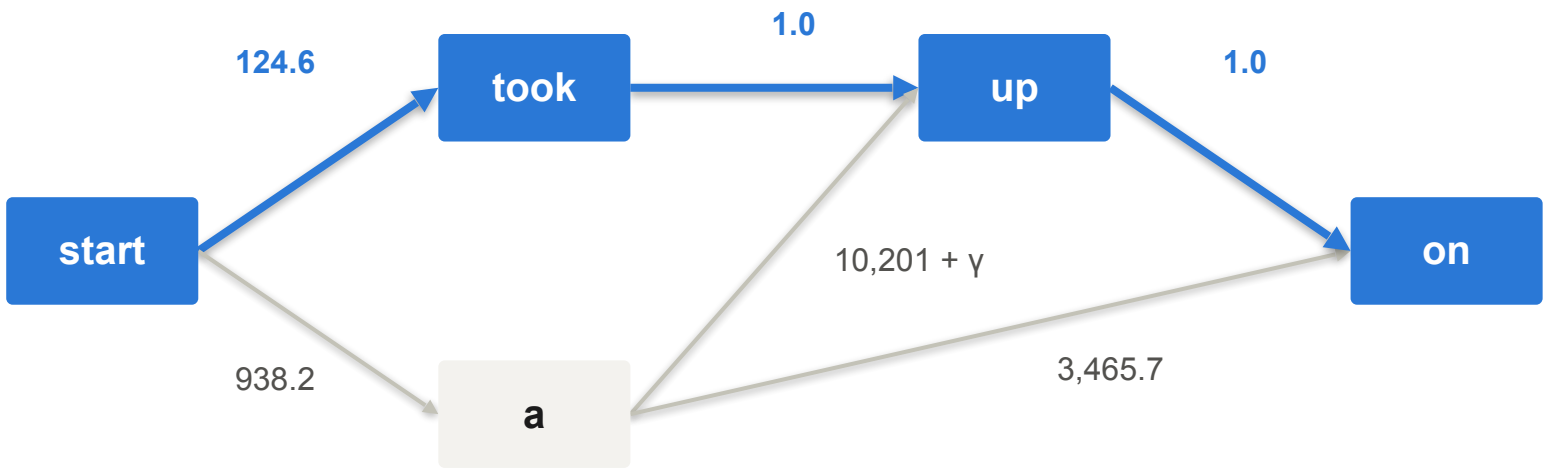
"up" and "on" each get a second candidate in a different class. Both cost more than the best entry plus γ, so they can never catch up and get dropped (gray).

If they were closer we would have to keep both, since the cheaper class might still pay γ later. Highlighted rows are the optimal path.

A WORKED EXAMPLE

Follow the pointers back to get the answer

EVERY FEASIBLE LINE, WITH ITS DEMERITS



126.6

smallest entry at the last break, in class 1

POINTERS, BACKWARD

on ← up ← took ← start

KNUTH-PLASS: 127 DEMERITS, 3 LINES

when she was bored she took
a golden ball, and threw it up
on

FIT AS MANY WORDS AS POSSIBLE: 4,404 DEMERITS, 2 LINES

when she was bored she took a
golden ball, and threw it up on

Taking the longest line first costs about 35× more here, because it forces tight lines.

Knuth & Plass, "Breaking Paragraphs into Lines," Software: Practice and Experience, 1981

THE ANALYSIS AND RESULTS

10 test cases, each twice the size of the last

10

paragraphs, from 2,048 to 1,048,576 words

2,353–1,216,218

legal breakpoints B

6.7 s

for the largest case

HOW I MADE THEM

Words sampled with a fixed seed from the paper's example paragraph, so lengths and hyphens stay realistic.

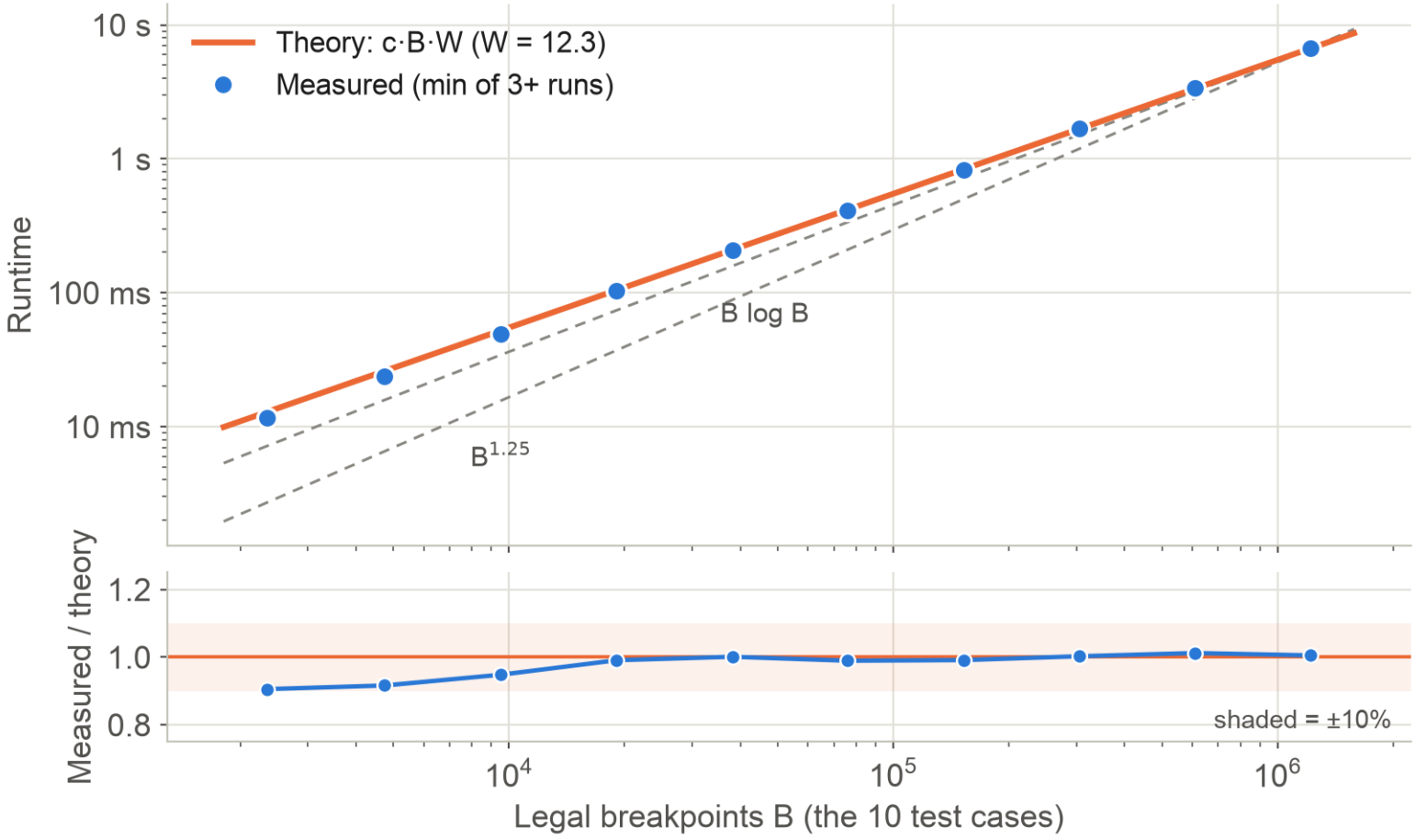
All use a 500-unit column and the paper's settings, so W stays at about 12.3 and only B changes.

HOW I KNOW THE ANSWERS ARE RIGHT

10 of 10	cases match a separate backward DP
21 of 21	line breaks match the paper's figures
2,748	small paragraphs match exhaustive search
10 of 10	bugs I planted on purpose get caught

Timed on an Apple M1 Max, Python 3.12, garbage collector off, minimum of at least 3 runs

Runtime grows linearly with B, just like the theory says



1.008

log-log slope on the 5 largest cases, 1.017 on all 10

1.97–2.09×

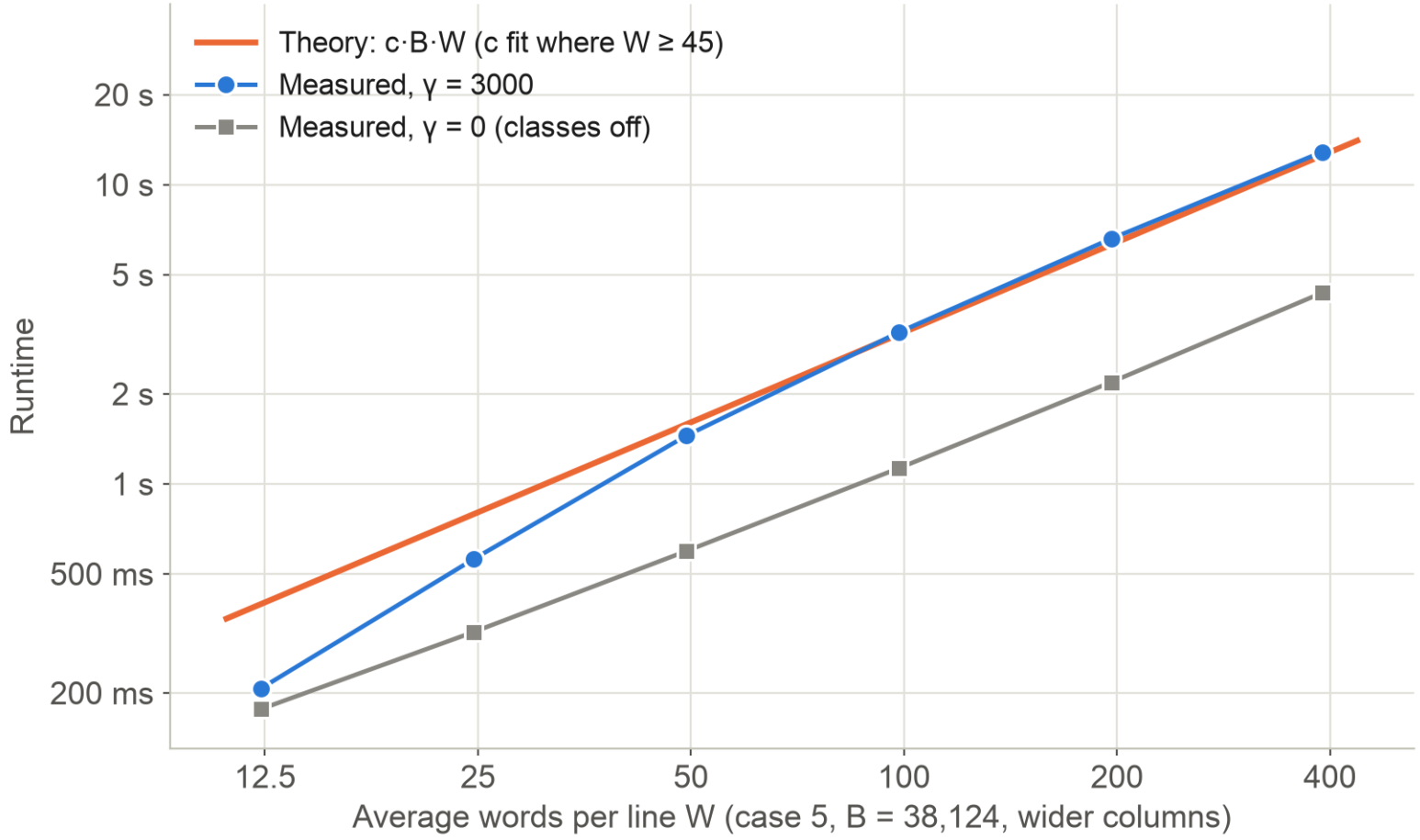
as long each time the size doubled

An extra log factor would show: B log B has a slope of 1.079 over these sizes.

Smallest cases run about 10% low. The first 250 words average 11 active nodes per break, not 18, and each check is a bit cheaper.

Log-log axes, so linear growth is a straight line with slope 1. Dashed lines go through the largest point. c fit on the 5 largest cases.

Runtime grows with W too, with one catch



1.054

slope once W reaches 45, but 1.193 overall

1.5 → 3.5

active nodes per W as lines get longer, then it levels off

That is the constant hiding in the O. Long lines let a break keep more of its 4 class nodes. Turn classes off (γ = 0) and the ratio stays at 1.2–1.3, with slope 0.93.

Slopes on log-log axes. Same text (case 5), wider and wider columns, so B stays at 38,124.

THE ANALYSIS AND RESULTS

My runtimes match O(B · W)

1.008	Slope against B with W fixed. Linear, and clearly not B log B.
1.054	Slope against W with B fixed, once the fitness class constant levels off.
281–302 ns	Cost of checking one active node across all 10 cases. Almost constant, so counting nodes explains the runtime.

Keep in mind: this is one machine, and W is the average words per line, not the max. The paper doesn't print its full font table, so I estimated some letter widths. Line breaks match its figures, but total demerits are 9% and 16% off.

Thanks! Questions?